OFFERLY

A PROJECT REPORT

Offerly: Campus Placement Application Tracker
Full Stack Development – II (24BET_TPP-603)

Submitted by

Yash Nalandwar
UID: 24BET10111

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Under the guidance of
Navdeep Kaur



Chandigarh University

November, 2026

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION 4
1.1 Introduction to Project: 4
1.2 Identification of Problem: 4
CHAPTER 2. BACKGROUND STUDY 5
2.1 Existing Solutions: 5
2.2 Problem Definition: 5
2.3 Goals/Objectives: 5
CHAPTER 3. DESIGN FLOW/PROCESS 6
3.1 Evaluation & Selection of Specifications/Features: 6
3.2 System Architecture: 7
3.3 Database Design: 7
3.4 REST API Design: 7
3.5 Design Flow: 8
3.6 Mapping with Lab Experiments: 8
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 10
4.1 Implementation of Solution: 10
4.2 Testing and Validation: 10
4.3 Output: 11
CHAPTER 5. CONCLUSION AND FUTURE WORK 18
5.1 Conclusion: 18
5.2 Future Work: 18
REFERENCES: 18

LIST OF FIGURES

Figure 1: Home page with search and live placement statistics 11
Figure 2: Job listing with search and filters 11
Figure 3: Registration page for students and the placement cell 12
Figure 4: Login page 12
Figure 5: Job details with the application form 13
Figure 6: Job details after applying, showing the application status 13
Figure 7: Student view: My Applications with status tracking 14
Figure 8: Application rejected by the CGPA eligibility rule 14
Figure 9: Placement-cell dashboard with statistics and job management 15
Figure 10: Admin form to publish a new job 15
Figure 11: Admin view of all applications with status updates 16
Figure 12: Dashboard in dark mode 16
Figure 13: Automated API test results (29 of 29 passed) 17

LIST OF TABLES

Table 1: Technology stack 6
Table 2: Three-tier architecture of Offerly 7
Table 3: Collections and schema validation 7
Table 4: REST API endpoints 7
Table 5: Coverage of lab experiments 1-9 8
Table 6: Selected test cases 10

CHAPTER 1. INTRODUCTION

1.1 Introduction to Project:

Offerly is a full-stack web application that digitises the campus placement process of a university. The Training and Placement Cell publishes job and internship opportunities, and students browse these opportunities, check whether they are eligible, apply online and follow the status of every application until the final result. The application is built on the MERN stack: React for the user interface, Node.js and Express.js for the REST API, and MongoDB with Mongoose for data storage. Users are authenticated with JSON Web Tokens (JWT) and their passwords are hashed with bcrypt.

The project was developed as the lab-based mini project (Experiment 10) of the course Full Stack Development – II (24BET_TPP-603). Its purpose is to bring together, in one real-world application, every concept practised in the earlier nine experiments: reusable React components, hooks, routing, interactive forms and tables, a Node.js backend, an Express REST API, JWT and bcrypt authentication, related MongoDB collections, and complete frontend-backend integration.

1.2 Identification of Problem:

In many colleges, placement drives are still announced through notice boards, messaging groups and e-mail, and applications are collected through spreadsheets or paper forms. This creates several problems. Students miss opportunities because announcements are scattered across channels. The placement cell spends a lot of time checking eligibility criteria such as minimum CGPA and branch by hand. Duplicate applications from the same student for the same job are common and have to be removed manually. Students have no single place to see whether they were shortlisted, called for an interview, selected or rejected.

There is therefore a need for a centralised, secure and easy-to-use system in which the placement cell can publish opportunities and manage applications, and in which students can apply once and track their progress in real time.

CHAPTER 2. BACKGROUND STUDY

2.1 Existing Solutions:

Commercial recruitment platforms such as LinkedIn, Naukri and Internshala, as well as college ERP modules, offer job listings and application tracking. However, general job portals are not designed around a single campus: they do not know a student's branch or CGPA, and they do not let a college placement cell control which students can apply. ERP modules are usually expensive, rigid and focused on administration rather than on the student experience. Many colleges therefore fall back on Google Forms and spreadsheets, which provide no authentication, no eligibility rules, and no protection against duplicate entries.

2.2 Problem Definition:

The objective of this project is to design and build a web application in which the placement cell (administrator) can create, update and delete job postings and manage the status of every application, and in which students can register, log in, search jobs and apply. The system must enforce the following business rule: a student cannot apply twice to the same job. It must also reject applications to jobs that are closed or past their deadline, and applications from students who do not meet the minimum CGPA or branch criteria.

2.3 Goals/Objectives:

The specific goals of this project are as follows:

- Develop a responsive React single-page application with reusable components and client-side routing.
- Build a RESTful backend with Node.js and Express.js that exposes GET, POST, PUT and DELETE endpoints.
- Design three related MongoDB collections (Users, Jobs, Applications) with Mongoose schema validation.
- Implement secure registration and login using bcrypt password hashing and JWT-based protected routes.
- Provide role-based access for students and placement-cell administrators.
- Enforce the business rule that a student cannot apply twice to the same job, at both API and database level.
- Integrate the frontend with the backend using fetch() so that every create, update and delete is reflected in the UI immediately.

CHAPTER 3. DESIGN FLOW/PROCESS

3.1 Evaluation & Selection of Specifications/Features:

The MERN stack was selected because a single language (JavaScript) is used across the whole application, JSON flows naturally from the database to the browser, and every technology in the stack was covered in the lab experiments. The technologies used are summarised in Table 1.

Table 1: Technology stack

Layer	Technology	Purpose in Offerly
Frontend	React 18, React Router 6, Vite	Component-based UI, page navigation, fast development build
Styling	CSS3 with custom properties	Responsive light and dark themes without a UI library
Backend	Node.js, Express.js 4	REST API, routing, middleware, central error handling
Database	MongoDB, Mongoose 8	Document storage, schemas, validation, relations with .populate()
Security	JSON Web Token, bcrypt	Stateless authentication and secure password hashing
Tools	Git, GitHub, VS Code, npm	Version control, editing and package management

Analysis of Features and Finalization Subject to Constraints

The main features of the application are:

- **Authentication:** student and placement-cell registration and login, with passwords hashed by bcrypt and sessions carried by a JWT.
- **Job management:** administrators create, edit, close and delete jobs with company, role, CTC/stipend, minimum CGPA, eligible branches and deadline.
- **Job discovery:** students search jobs by company, role or location and filter by type and status.
- **Applications:** students apply with an optional cover note and resume link, view all their applications, and withdraw an application while it is still pending.
- **Status tracking:** administrators move each application through applied, shortlisted, interview, selected or rejected, and can add a remark.
- **Dashboard:** live counts of open jobs, total applications, interviews and selections.
- **Business rules:** no duplicate applications, no applications to closed or expired jobs, and CGPA/branch eligibility checks.

Because the project had to be completed within the lab schedule, features such as file upload of resumes, e-mail notifications and interview scheduling were left out. The focus was kept on correct data modelling, security and complete CRUD integration.

3.2 System Architecture:

Offerly follows a three-tier client-server architecture. The React client runs in the browser and talks to the Express server only through JSON over HTTP. The Express server validates the JWT on protected routes, applies the business rules, and uses Mongoose models to read and write MongoDB. In production the same Express server also serves the built React files, so the whole application is deployed as one web service.

Table 2: Three-tier architecture of Offerly

Tier	Component	Responsibility
Presentation	React SPA (pages, components, AuthContext)	Renders UI, keeps form state, calls the API with fetch() and the JWT
Application	Express routes and middleware	protect / authorize middleware, validation, business rules, JSON responses
Data	Mongoose models and MongoDB	Schemas, validators, unique indexes, references between collections

3.3 Database Design:

The database contains three related collections. Applications reference both the student (User) and the job (Job), and every Job references the administrator who posted it. Mongoose .populate() is used to return the referenced documents together with an application.

Table 3: Collections and schema validation

Collection	Important fields	Relations and validation
Users	name, email, password, role, rollNo, branch, cgpa	email unique and validated by regex; password min 6 chars, hashed, never returned; role is student or admin; CGPA 0-10
Jobs	company, title, description, type, ctc, location, minCgpa, eligibleBranches, deadline, status	postedBy references Users; required fields; type and status enums; CTC cannot be negative
Applications	student, job, status, coverNote, resumeLink, adminRemark	student references Users, job references Jobs; status enum; compound unique index on (student, job)

3.4 REST API Design:

Table 4: REST API endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a student (or admin with the admin code)
POST	/api/auth/login	Public	Verify credentials and return a JWT
GET	/api/auth/me	Logged in	Current user profile (protected route)

Method	Endpoint	Access	Description
PUT	/api/auth/me	Logged in	Update profile, CGPA or password
GET	/api/jobs	Public	List jobs with search and filters
GET	/api/jobs/:id	Public	Details of one job
POST	/api/jobs	Admin	Create a job
PUT	/api/jobs/:id	Admin	Update a job
DELETE	/api/jobs/:id	Admin	Delete a job and its applications
POST	/api/applications	Student	Apply to a job (duplicate returns 409)
GET	/api/applications/me	Student	My applications with job details
GET	/api/applications	Admin	All applications with student and job details
PUT	/api/applications/:id	Owner / Admin	Change status or edit a pending application
DELETE	/api/applications/:id	Owner / Admin	Withdraw or remove an application

3.5 Design Flow:

The flow of data when a student applies for a job is as follows:

1. The student logs in; the server checks the password with bcrypt and returns a signed JWT, which React stores and sends in the Authorization header of later requests.
2. The student opens a job and clicks "Apply now"; React sends POST /api/applications with the job id, cover note and resume link.
3. The protect middleware verifies the JWT and the authorize middleware confirms that the user is a student.
4. The route checks that the job is open and before its deadline, and that the student meets the minimum CGPA and branch criteria.
5. The route checks for an existing application by the same student for the same job and returns HTTP 409 if one exists.
6. Mongoose validates the new document and saves it; the unique (student, job) index guarantees that even simultaneous requests cannot create a duplicate.
7. The saved application is returned with the job populated, and React updates the page to show the application status without reloading.

3.6 Mapping with Lab Experiments:

Table 5: Coverage of lab experiments 1-9

Exp.	Concept	Implementation in Offerly
1	React components and JSX	Reusable Navbar, JobCard, StatusBadge, StatCard, JobForm, Alert, Loader; lists rendered with map()
2	React hooks	useState for all controlled forms; useEffect for fetching jobs,

Exp.	Concept	Implementation in Offerly
		applications and stats
3	React Router	Home, Jobs, Job details, Login, Register, My Applications, Profile and Admin pages; useParams and useSearchParams
4	Interactive frontend	Search box, filters, status pills, cards and tables, role-based conditional rendering
5	Node.js	Server entry point, environment configuration, seed script
6	Express REST API	GET, POST, PUT and DELETE routes for jobs and applications with error middleware
7	JWT and bcrypt	Registration, login, protect and authorize middleware, ProtectedRoute in React
8	MongoDB and Mongoose	Three related collections, schema validation, .populate(), unique compound index
9	Frontend-backend integration	fetch() wrapper with JWT; UI state updated after every create, update and delete

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution:

The application was implemented as two Node.js projects in one repository: a server folder containing the Express API and Mongoose models, and a client folder containing the React application built with Vite. During development the Vite server forwards every /api request to Express. For production, the React app is built into static files that Express serves, so the application runs as one service connected to MongoDB Atlas.

Passwords are hashed by a Mongoose pre-save hook using bcrypt with 10 salt rounds and are excluded from every query result. On login the server issues a JWT signed with a secret stored in an environment variable. The React AuthContext keeps the logged-in user, restores the session on page reload, and the ProtectedRoute component redirects visitors who are not logged in or do not have the required role.

The business rule "a student cannot apply twice to the same job" is enforced in three layers: the database has a compound unique index on student and job, the API checks for an existing application and returns HTTP 409 Conflict with a clear message, and the user interface replaces the apply form with the current application status once the student has applied.

4.2 Testing and Validation:

An automated script exercised the API end to end with 29 test cases, covering registration, login, invalid credentials, validation errors, access without a token (401), access with the wrong role (403), search, full CRUD on jobs and applications, .populate() results, deleting a job together with its applications, and the business rules. All 29 tests passed (Figure 13). In particular, when three identical apply requests were sent at the same moment, exactly one was accepted and the others were rejected, which shows that the unique index protects the rule even under concurrent requests.

Table 6: Selected test cases

Test scenario	Expected result	Observed
Register with an e-mail that already exists	409 Conflict	Pass
Open a protected route without a token	401 Unauthorized	Pass
Student tries to create a job	403 Forbidden	Pass
Student applies to the same job twice	409 "You have already applied to this job"	Pass
Three identical apply requests at the same time	Exactly one application created	Pass
Apply to a closed or expired job	400 Bad Request	Pass
Student below the minimum CGPA applies	400 with eligibility message	Pass

Test scenario	Expected result	Observed
Withdraw after being shortlisted	400 Bad Request	Pass
Admin deletes a job	Job and its applications removed	Pass

4.3 Output:

The screenshots below show the main screens of the application.

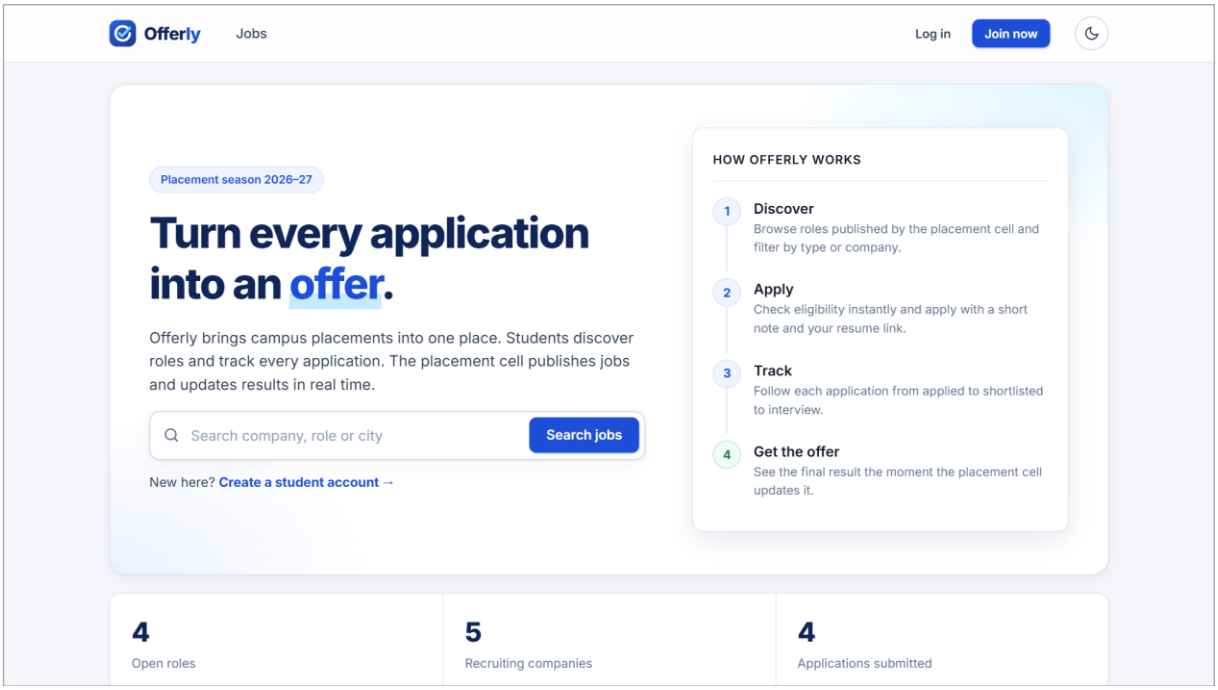


Figure 1: Home page with search and live placement statistics

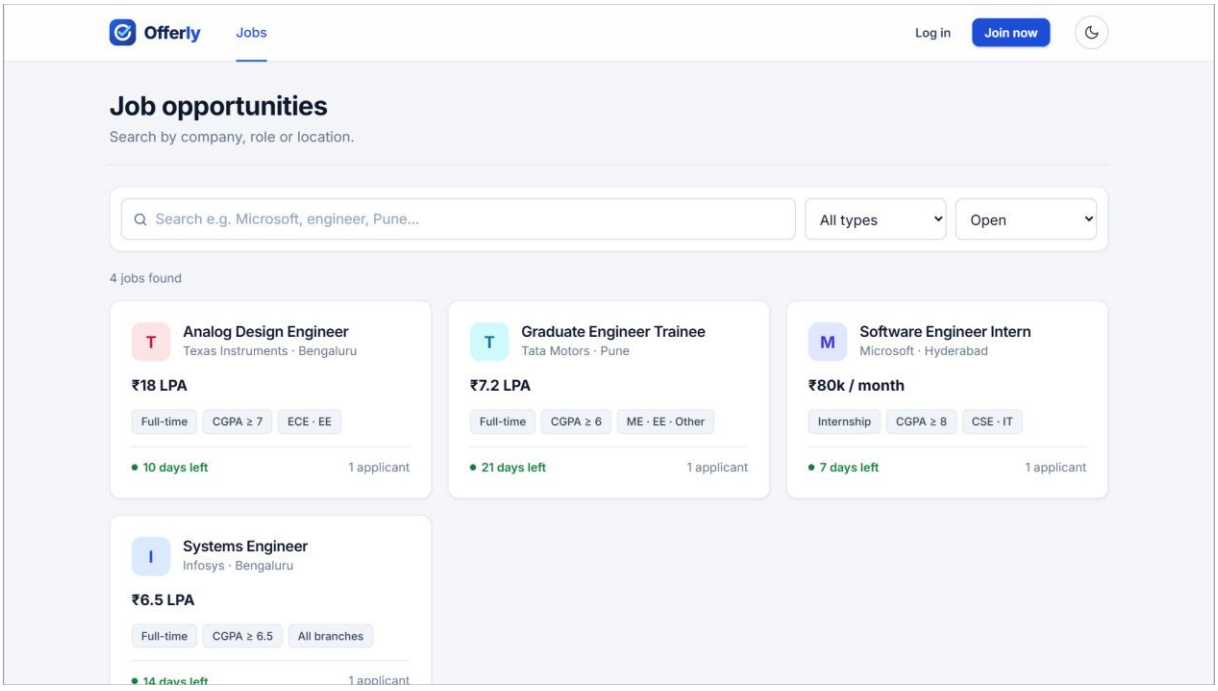


Figure 2: Job listing with search and filters

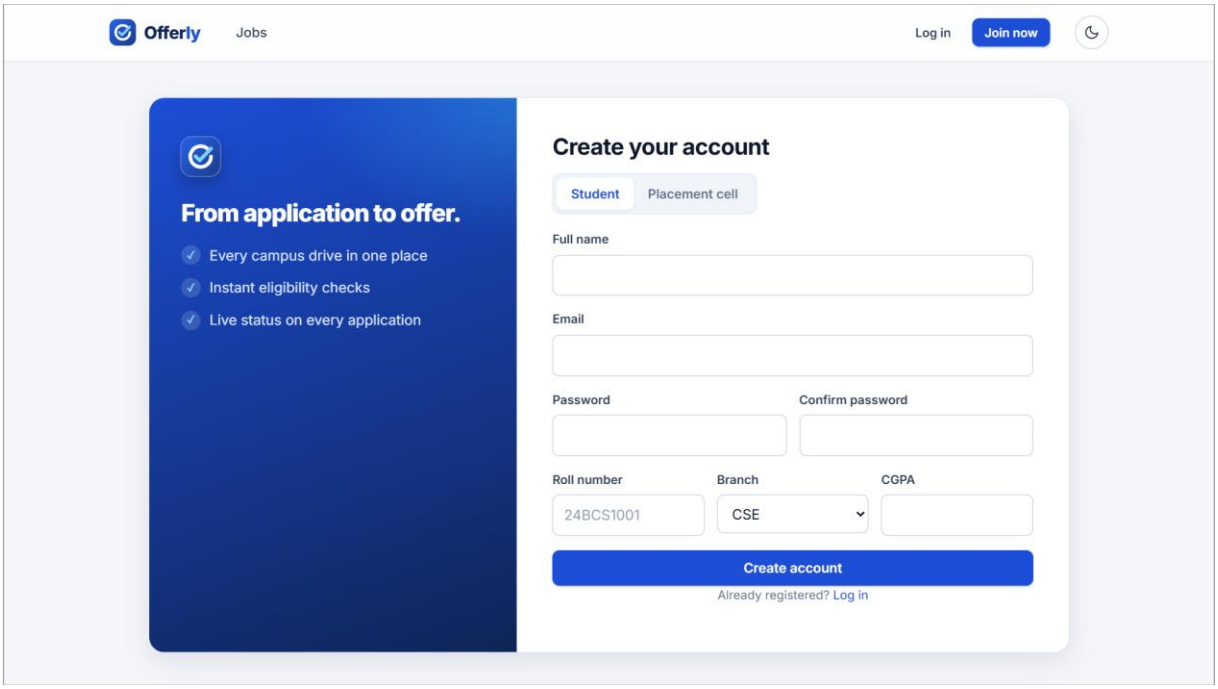


Figure 3: Registration page for students and the placement cell

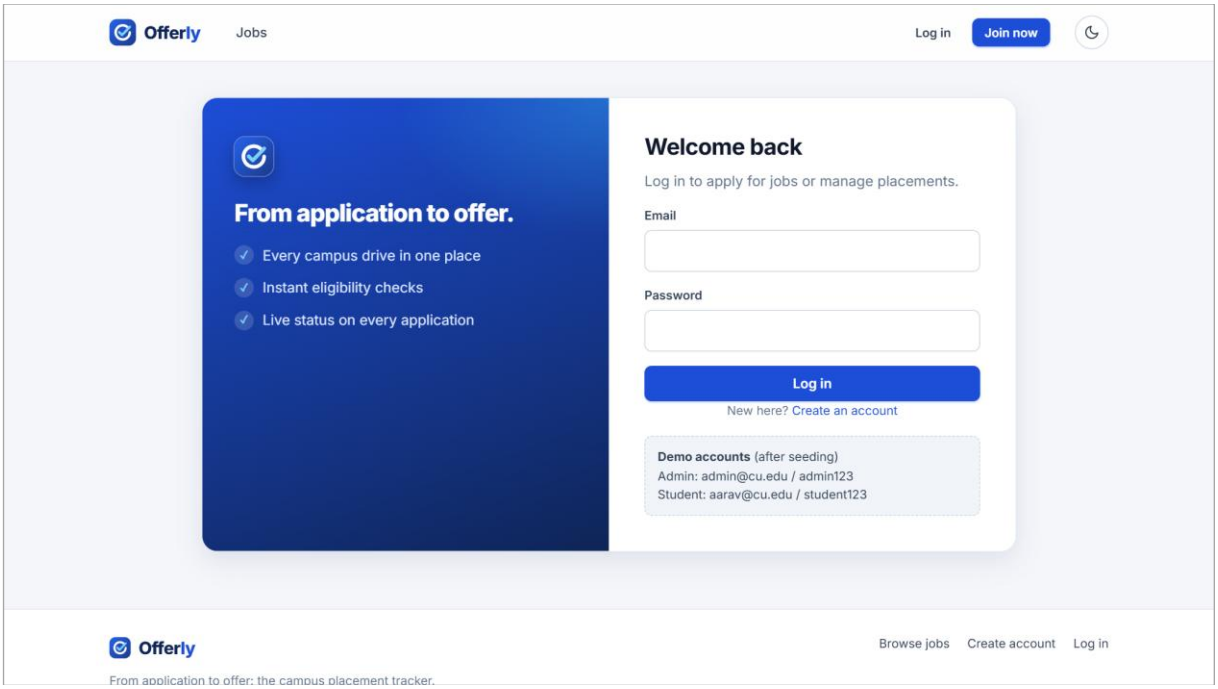


Figure 4: Login page

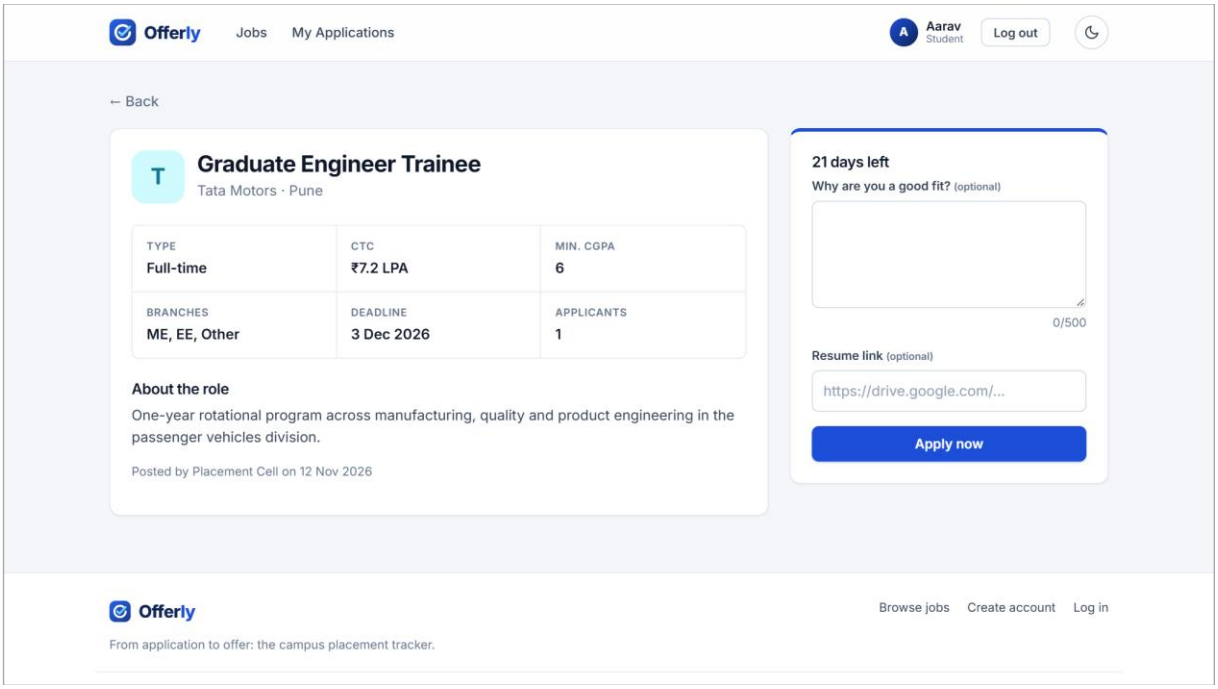


Figure 5: Job details with the application form

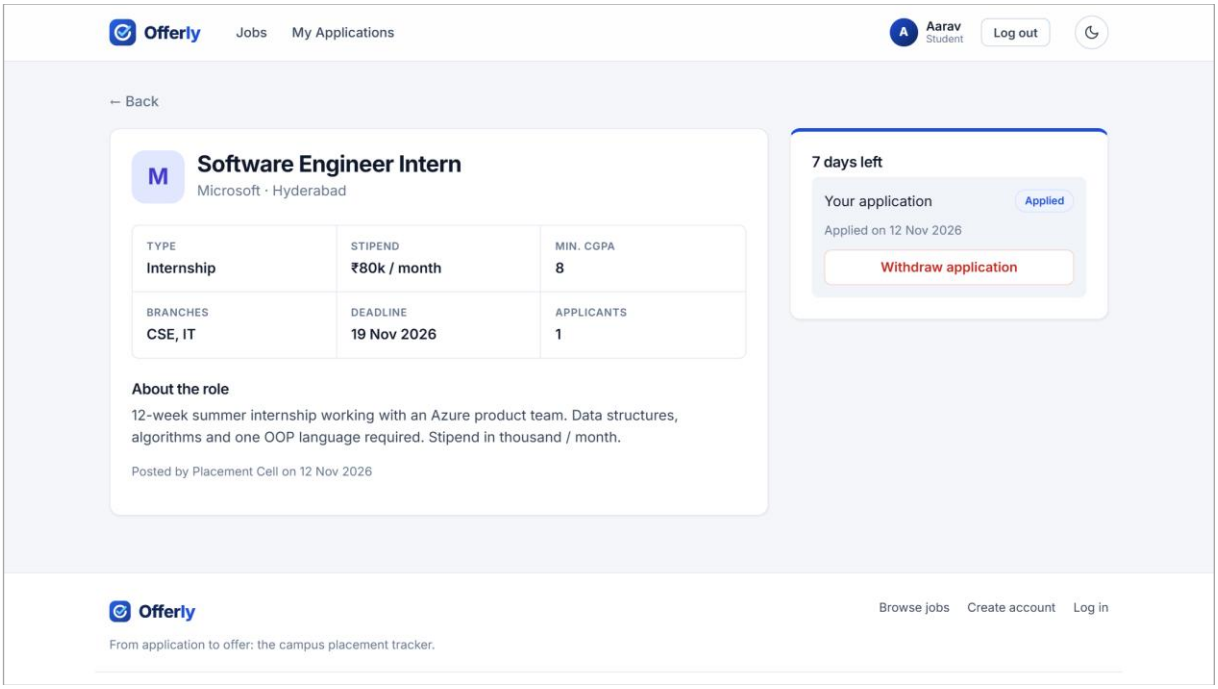


Figure 6: Job details after applying, showing the application status

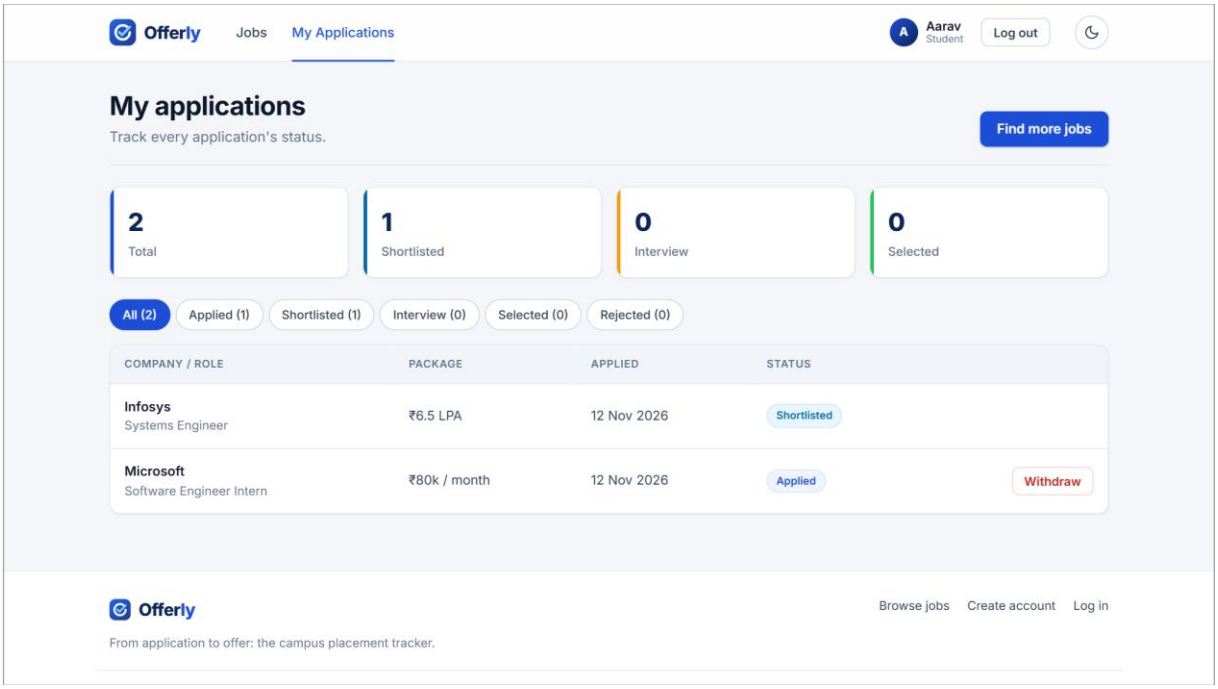


Figure 7: Student view: My Applications with status tracking

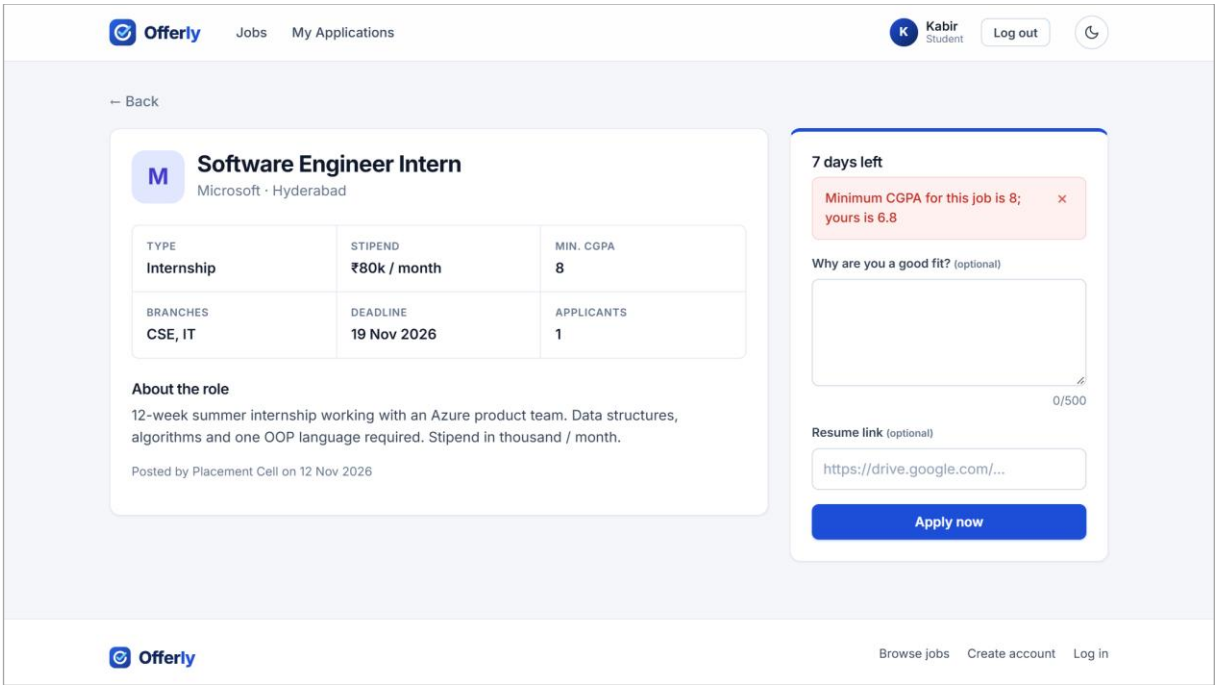


Figure 8: Application rejected by the CGPA eligibility rule

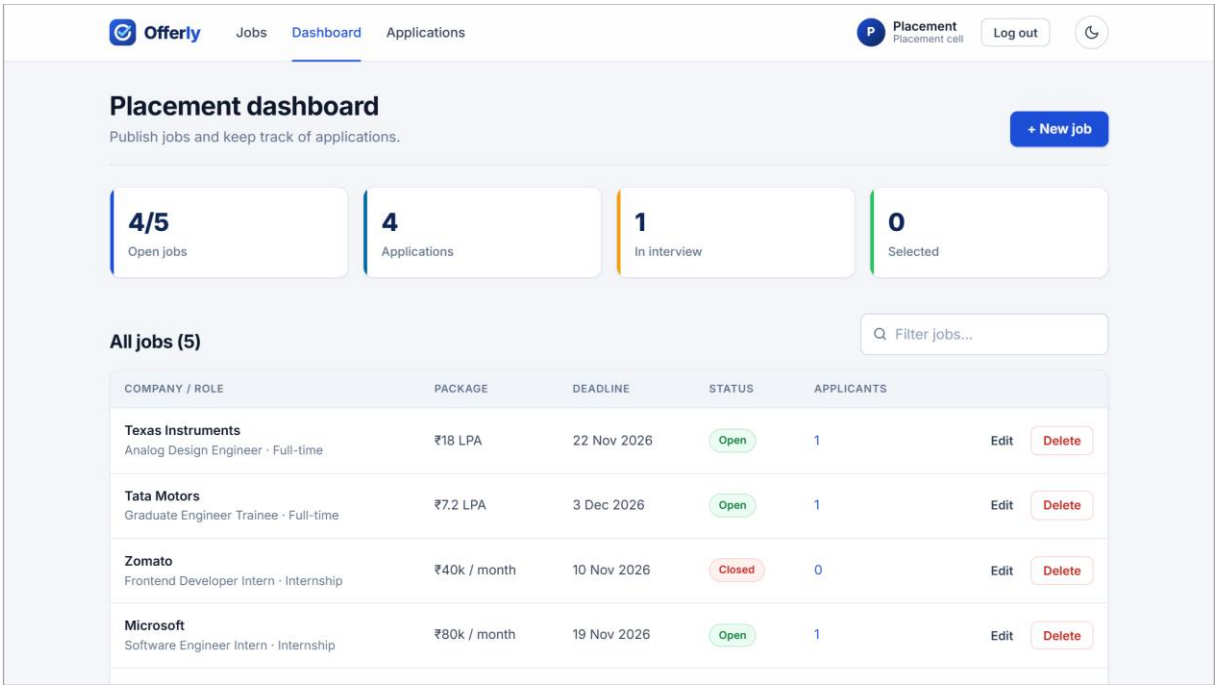


Figure 9: Placement-cell dashboard with statistics and job management

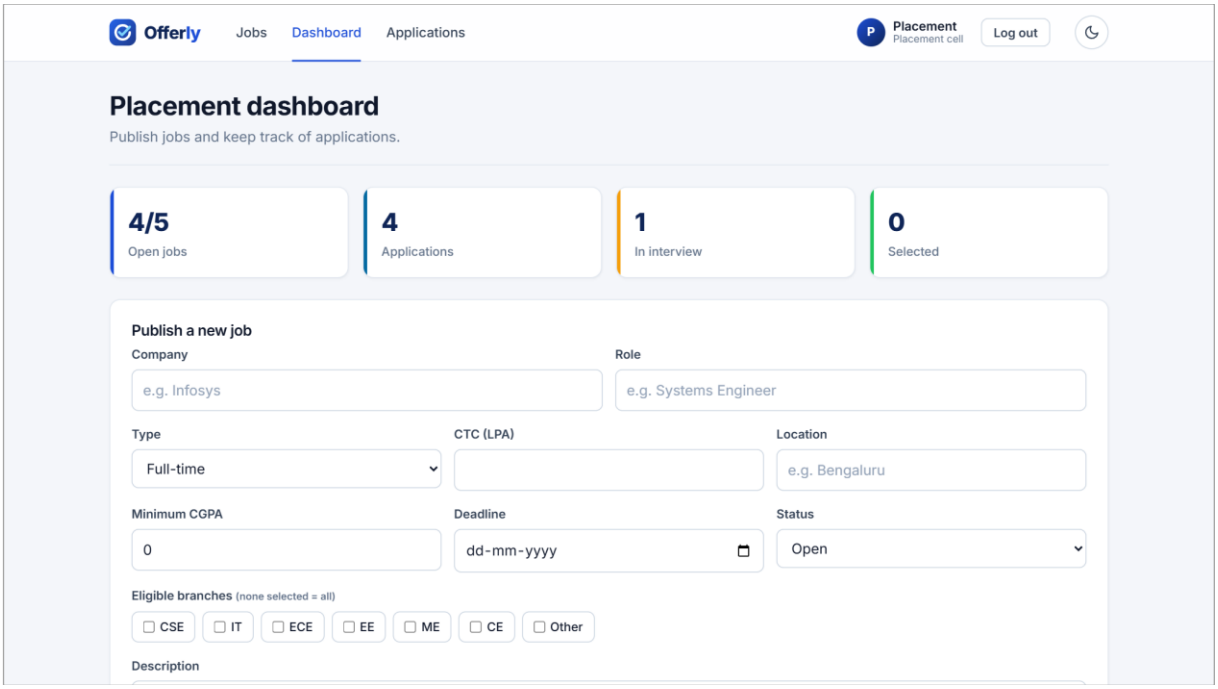


Figure 10: Admin form to publish a new job

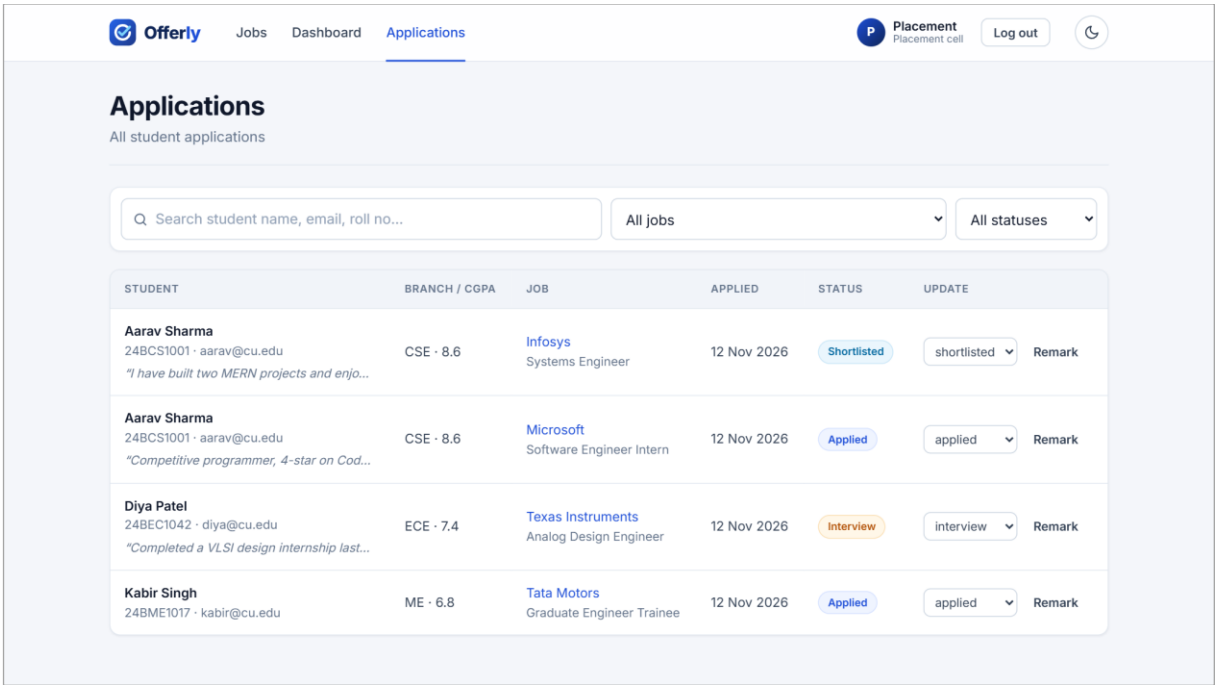


Figure 11: Admin view of all applications with status updates

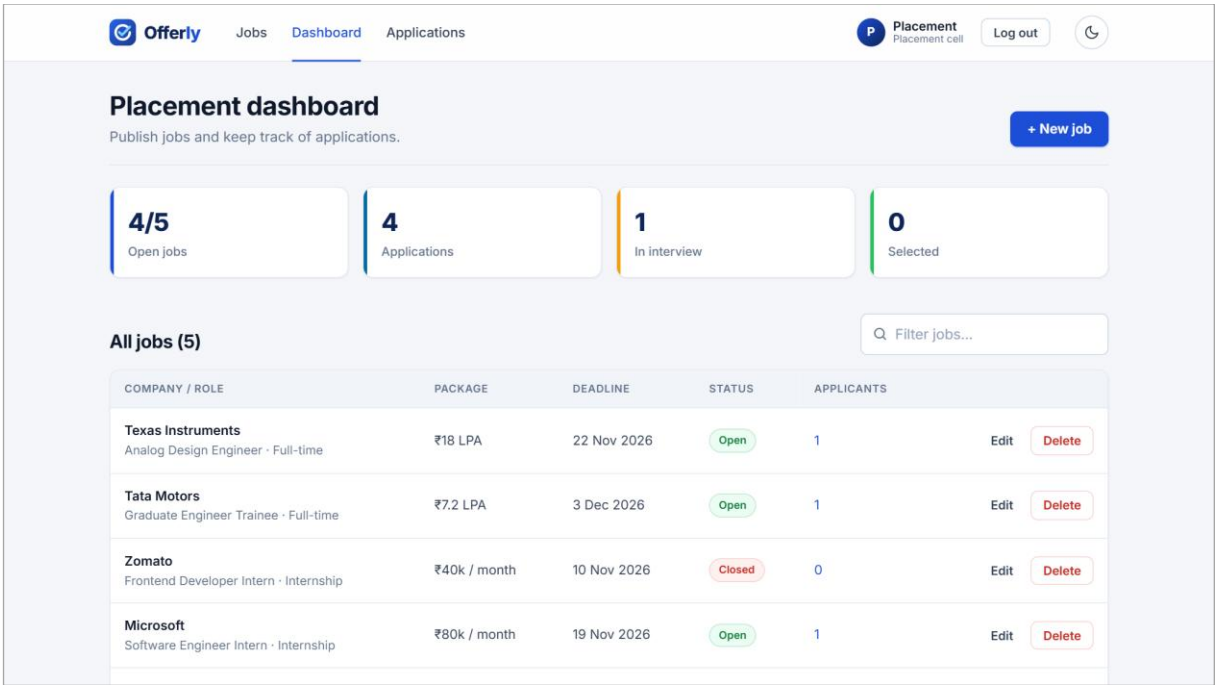


Figure 12: Dashboard in dark mode

```
PS D:Projects\sd> node apitest.mjs
PASS  admin login
PASS  wrong password 401
PASS  register 201
PASS  duplicate email 409
PASS  validation 400
PASS  bad admin code 400
PASS  protected without token 401
PASS  me with token
PASS  search jobs
PASS  filter type
PASS  student cannot create job 403
PASS  admin creates job
PASS  job validation 400
PASS  admin updates job (PUT)
PASS  student applies
PASS  BUSINESS RULE: duplicate apply 409
PASS  BUSINESS RULE: concurrent duplicates -> exactly one 201
PASS  closed job rejected
PASS  ineligible (CGPA/branch) rejected
PASS  my applications
PASS  student cannot list all 403
PASS  admin lists with populate
PASS  admin updates status
PASS  invalid status 400
PASS  cannot withdraw after shortlist 400
PASS  other student cannot touch 403
PASS  stats
PASS  admin deletes job + cascades
PASS  bad id 400

29 passed, 0 failed
```

Figure 13: Automated API test results (29 of 29 passed)

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion:

Offerly successfully demonstrates how React, Node.js, Express.js, MongoDB, Mongoose and JWT-based authentication work together in a complete, real-world web application. The system replaces scattered placement announcements and spreadsheets with a single secure platform where the placement cell manages jobs and applications and students apply and track their progress. All of the required functionality was implemented and validated: registration and login with bcrypt and JWT, protected and role-based routes, full CRUD operations, three related collections with schema validation and .populate(), and the business rule that a student cannot apply twice to the same job. The project provided valuable hands-on experience in designing REST APIs, modelling data, securing applications and integrating a modern frontend with a backend.

5.2 Future Work:

The project can be further enhanced by adding the following features:

- Resume upload to cloud storage instead of a resume link.
- E-mail and in-app notifications when an application status changes.
- Interview scheduling with calendar invitations for shortlisted students.
- Analytics for the placement cell, such as branch-wise and company-wise placement statistics and export to Excel.
- Company accounts so that recruiters can post jobs and review applicants directly.
- Automated testing in a CI pipeline and deployment with Docker.

These improvements would make Offerly suitable for use by a real university placement cell across multiple placement seasons.

REFERENCES:

1. React Documentation, https://react.dev
2. React Router Documentation, https://reactrouter.com
3. Express.js Documentation, https://expressjs.com
4. Mongoose Documentation, https://mongoosejs.com/docs
5. MongoDB Manual, https://www.mongodb.com/docs/manual
6. JSON Web Token (RFC 7519), https://datatracker.ietf.org/doc/html/rfc7519
7. Node.js Documentation, https://nodejs.org/docs